

★ PROJECT REPORT ★

ON

TWITTER TIMELINE LINKED LIST MANAGEMENT SYSTEM

A Twitter-Style Linked List Engine



INSERT TWEET



DELETE TWEET



SEARCH TWEET



TRAVERSE TIMELINE



REVERSE LIST



COUNT NODES

LINKED LIST REPRESENTATION



"Dynamic • Efficient • Scalable"



SUBMITTED BY

Divyansh



COURSE : BCA



SEMESTER : II (Second Semester)



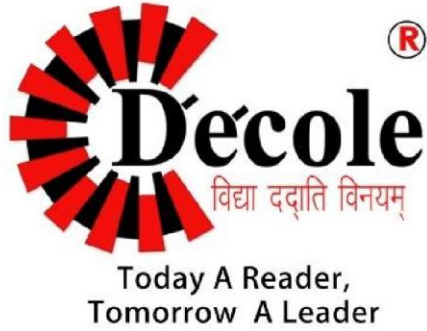
SESSION : 2025-26



SUBJECT : Data Structures with C++



COLLEGE : Dezyne École College



Dezyne École College
www.dezyneecole.com

Title of Project

Twitter Timeline

Subject Name

C++ programming and data structure

Submitted By

Divyansh

Department – BCA

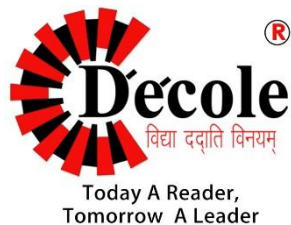
Semester – 2nd

Session - 2025-26

Project Report of
C++ programming and data structure

Submitted to
Dezyne École College
Towards the Partial Fulfilment of
2nd Semester

By
Divyansh



www.dezyneecole.com 2025-2026

Acknowledgement

We take this opportunity to express our sincere gratitude to Dezyne École College for providing us with the knowledge, resources, and guidance essential to the successful completion of this project.

Our deepest thanks to our respected Principal, Dr. Vinita Mathur, for her visionary leadership and continuous encouragement. Her commitment to academic excellence and student growth has been a constant source of motivation and pride.

We are equally thankful to all our faculty members for their consistent support, expert advice, and valuable insights that shaped our learning and helped us achieve this milestone.

Lastly, we extend heartfelt thanks to everyone who directly or indirectly contributed to the success of this project.

We are proud to be part of Dezyne École College, an institution that truly nurtures professionalism, innovation, and purpose.

With sincere appreciation,

Divyansh

Department of BCA

Dezyne École College, Ajmer

(Right-click → Update Field in Word to refresh page numbers.)

Bachelor of Computer Applications (BCA).....	Error! Bookmark not defined.
1. Introduction	7
Why this project matters	7
Project objectives	7
2. What is a Linked List?	8
Two main variants	8
3. Real-World Context — Twitter Timeline	9
Real-life mapping	9
Operation → Twitter feature mapping	9
4. Singly vs Doubly Linked List — At a Glance	10
5. Why Not an Array?	11
6. Node Structure & Memory Representation	12
C++ struct definition (Singly)	12
Memory layout — nodes scattered in RAM	12
7. All Linked List Operations — Overview	13
8. Insert Operations	14
8.1 Insert at Head — $O(1)$	14
Step-by-step algorithm	14
8.2 Insert at Tail — $O(1)$ with tail pointer	14
8.3 Insert at Position k — $O(k)$	15
Trace	15
9. Delete Operations	16
9.1 Delete Head — $O(1)$	16
9.2 Delete Tail — $O(n)$ in Singly	16
9.3 Delete by Value — $O(n)$	16
10. Traversal — Forward & Backward	18
10.1 Forward Traversal (Singly & Doubly) — $O(n)$	18
10.2 Backward Traversal (Doubly only) — $O(n)$	18
11. Search Operations	19
Why binary search fails on linked lists	19
12. Count Nodes	20
13. Reverse a Linked List — $O(n)$ Time, $O(1)$ Space	21
Three-pointer algorithm	21
Trace on $[T4] \rightarrow [T3] \rightarrow [T2] \rightarrow [T1] \rightarrow \text{NULL}$	21
14. Doubly Linked List Operations	22

DLL Node structure	22
14.1 Insert at Head — $O(1)$	22
14.2 Insert at Tail — $O(1)$	22
14.3 Delete Tail — $O(1)$ ★ key advantage	23
14.4 Delete Any Node N (pointer known) — $O(1)$	23
15. Complete Complexity Comparison	24
Memory comparison	24
16. Main Project — MainProjectLinkedList.cpp	25
Source code (1/2)	25
Source code (2/2)	26
Sample run	27
17. Singly Linked List Project — MyLinkedList.cpp	28
Source code (1/2)	28
Source code (2/2)	29
Sample output	30
18. Doubly Linked List Project — MyDoubly_linklist.cpp	31
Source code (1/2)	31
Sample output	32
19. Real-World Applications	33
20. Advantages & Disadvantages	34
Advantages	34
Disadvantages	34
21. Conclusion	35
22. Bibliography / References	36
A. Appendix — Source Code Index	37

1. Introduction

Modern social-media platforms such as **Twitter** handle millions of incoming posts every second. The home timeline that every user sees is a dynamic, ever-changing list — a new tweet must appear instantly at the top, an offending tweet must disappear in a single click, and the rest of the feed must scroll smoothly from newest to oldest. The data structure that quietly powers this experience is the humble **Linked List**.

This project re-creates that timeline mechanism in **C++** using two flavours of linked lists — the **Singly Linked List** and the **Doubly Linked List** — and combines them into a single **Linked-List Management System**. Three connected programs are presented:

- Part 1 — Main Project: A menu-driven console application that exposes every linked-list operation (MainProjectLinkedList.cpp).
- Part 2 — Singly Linked List: Focused implementation of all singly-linked operations (MyLinkedList.cpp).
- Part 3 — Doubly Linked List: Forward + backward traversal and $O(1)$ deletions (MyDoubly_linklist.cpp).

Why this project matters

Linked lists are the gateway to every advanced data structure — trees, graphs, hash chains, LRU caches and adjacency lists are all built on top of pointer-based nodes. Mastering them in the context of a real-world product (Twitter) makes the theory unforgettable.

Project objectives

- Build a working, menu-driven Twitter-timeline simulator in C++.
- Implement and explain every fundamental linked-list operation.
- Compare Singly vs Doubly variants in time and space.
- Produce exam-ready notes, complexity tables and viva answers.

2. What is a Linked List?

A **Linked List** is a linear data structure where elements (called **nodes**) are not stored in contiguous memory locations. Each node holds its data and one or two **pointers** — memory addresses — that link it to neighbouring nodes. The nodes can live anywhere in RAM; the pointers stitch them into a logical sequence.

Visual: a Singly Linked List of tweets

head	T1 (newest)	T2	T3	T4 (oldest)	NULL
→	→	→	→	tail	

Definition (exam-ready)

A linked list is a collection of self-referential nodes in which each node contains data and a pointer to the next node, allowing dynamic, non-contiguous storage.

Two main variants

Singly Linked List (SLL) — each node has 1 pointer (next). Forward traversal only.

Doubly Linked List (DLL) — each node has 2 pointers (prev + next). Both forward and backward traversal.

Visual: a Doubly Linked List of tweets

NULL	T1	T2	T3	T4	NULL
← head	↔	↔	↔	tail →	

3. Real-World Context — Twitter Timeline

Twitter's home timeline shows tweets in **reverse-chronological order** — the newest at the top, the oldest at the bottom. Three operations dominate this system:

- Insert at head: a new tweet goes at the very top — must be instant — $O(1)$.
- Delete from any position: content moderation removes any tweet immediately — $O(1)$.
- Traverse from head to tail: displaying the timeline scrolls newest→oldest — $O(n)$.

These three needs define the Linked List — a data structure that does all three efficiently. The singly variant covers the basic feed; the doubly variant unlocks features like “see older replies first” and instant tail deletion.

Real-life mapping

Tweet = node data | **Next pointer** = link to the older tweet | **Head** = newest tweet on screen | **Tail** = oldest loaded tweet at the bottom of scroll.

Operation → Twitter feature mapping

Linked-List Operation	Twitter Feature	Frequency
Insert at head	Posting a new tweet	Extremely high
Delete head	Undoing your latest tweet	Low
Delete any node	Moderation / user deletes tweet	Medium
Forward traversal	Scrolling down the feed	Highest
Backward traversal	Scroll-to-top / oldest reply	Medium (DLL only)
Search by keyword	Find a tweet by hashtag	Medium
Reverse	Show oldest first toggle	Low (rare)

4. Singly vs Doubly Linked List — At a Glance

Before implementation, it is important to know which variant is best suited for a given scenario. The table below compares the two on the parameters that actually matter at the BCA syllabus level.

Property	Singly Linked List	Doubly Linked List
Pointers per node	1 (next only)	2 (prev + next)
Traversal direction	Forward only (head → tail)	Both directions
Delete tail	$O(n)$ — must scan to 2nd-last node	$O(1)$ — use tail.prev
Delete any node (with pointer)	$O(n)$ — scan for predecessor	$O(1)$ — both neighbours known
Extra memory per node	Less (1 pointer $\times$ n)	More (2 pointers $\times$ n = +8 bytes/node)
Implementation effort	Easier — fewer pointer rewires	Harder — both prev and next must be updated
Typical use	Stacks, hash chaining, simple queues	Timelines, browsers, LRU cache, dequeues

Key takeaway

Use a Singly Linked List when memory is tight and you only ever move forward. Choose a Doubly Linked List the moment you need bidirectional traversal or constant-time deletion of arbitrary nodes — exactly the case for a Twitter timeline.

5. Why Not an Array?

Arrays are simpler, faster to access by index and cache-friendly — yet Twitter cannot use a plain array for its timeline. The reason becomes obvious once you compare the two structures on the four operations that the timeline performs continuously.

Operation	Array	Linked List	Twitter's need
Insert at front	$O(n)$ — shift every element right	$O(1)$ — rewire 2 pointers	New tweet on top → instant
Delete any element	$O(n)$ — shift remaining left	$O(1)$ — rewire neighbours	Remove tweet immediately
Access by index	$O(1)$ — direct formula	$O(n)$ — traverse from head	Not needed for sequential feed
Dynamic size (grow)	Needs reallocation	Trivial — just add a node	Tweet count is unpredictable
Search (unsorted)	$O(n)$ linear	$O(n)$ linear only	Both require traversal

Conclusion

Arrays win at random access; linked lists win at insertion/deletion. A Twitter timeline performs many more insertions/deletions than random index lookups, so the linked list is the correct choice.

6. Node Structure & Memory Representation

In a singly linked list each node stores data and ONE pointer — *next* — which holds the memory address of the following node. The last node's *next* = NULL. We maintain a **head** pointer to always find the start, and optionally a **tail** pointer for efficient end-access.

C++ struct definition (Singly)

```
Node.cpp // g++ -std=c++17
1  class NODE
2  {
3  public:
4      string ID;
5      string UNAME;
6      string TWEET;
7      NODE *NEXT;
8      NODE(string x, string y, string z){
9          ID = x;
10         UNAME = y;
11         TWEET = z;
12         NEXT = nullptr;
13     }
14 };
```

Memory layout — nodes scattered in RAM

```
memory.txt
1  head -> [Tweet4 | 0x3A10] -> [Tweet3 | 0x1F40] -> [Tweet2 | 0x7C80] ->
   [Tweet1 | NULL]
2      at 0x2B00          at 0x3A10          at 0x1F40
   at 0x7C80
3      ^ newest
   oldest (tail)
4
5  Nodes are SCATTERED in RAM - not consecutive like an array.
6  The only way to reach Tweet2 is: start at head, follow next, follow next,
   arrive.
```

Pointer in plain English

A pointer is just a variable that stores the address of another variable. When we write `node->next`, we are saying: “go to the memory address kept inside `next` and look at the node that lives there.”

7. All Linked List Operations — Overview

Every linked-list program — from a simple stack to a Twitter timeline — is built from a small set of fundamental operations. The remainder of this report explains each of them in depth, with algorithms, code, traces and complexity.

#	Operation	Singly	Doubly	Used in Twitter for
1	Insert at Head	$O(1)$	$O(1)$	Posting a new tweet
2	Insert at Tail	$O(1)^*$	$O(1)$	Loading older tweets
3	Insert at Position k	$O(k)$	$O(k)$	Pinning a tweet at position k
4	Delete Head	$O(1)$	$O(1)$	Undo last tweet
5	Delete Tail	$O(n)$	$O(1)$	Trim oldest from cache
6	Delete by Value	$O(n)$	$O(n)$	Moderation
7	Forward Traversal	$O(n)$	$O(n)$	Scroll feed top→bottom
8	Backward Traversal	—	$O(n)$	Show oldest reply first
9	Search by Keyword	$O(n)$	$O(n)$	Hashtag search
10	Count Nodes	$O(1)/O(n)$	$O(1)/O(n)$	Total tweets in cache
11	Reverse	$O(n)$	$O(n)$	Toggle "oldest first"

* $O(1)$ only when a tail pointer is maintained; $O(n)$ otherwise.

8. Insert Operations

8.1 Insert at Head — $O(1)$

The newest tweet always goes at the top of the timeline. Head insertion achieves this in exactly **2 pointer operations** — always $O(1)$ regardless of how many tweets already exist.

Visual — before & after

Before	head → [B] → [C] → NULL
After	head → [A] → [B] → [C] → NULL

Time: $O(1)$ | Pointer ops: 2

insertAtHead.cpp // g++ -std=c++17

```

1 void insertAtHead(string text, string user) {
2     TweetNode* newNode = new TweetNode(text, user);
3
4     if (head == nullptr) {           // empty list
5         head = tail = newNode;
6     } else {
7         newNode->next = head;        // step 2: Link forward
8         head = newNode;              // step 3: advance head
9     }
10    size++;
11 }
```

Complexity

Time: $O(1)$ — independent of list length. Space: $O(1)$ extra (one new node).

Step-by-step algorithm

- Create new node: `TweetNode* newNode = new TweetNode(text, user);` — allocates heap memory, sets `next = NULL`.
- Connect forward: `newNode->next = head;` — new node now points to the current head.
- Move head pointer: `head = newNode;` — head now refers to the new node.
- Handle empty list: if list was empty, set `tail = newNode;`
- Increment counter: `size++;`

8.2 Insert at Tail — $O(1)$ with tail pointer

Loading older tweets at the bottom of the feed appends nodes at the tail. If we maintain a tail pointer, this is also $O(1)$.

insertAtTail.cpp // g++ -std=c++17

```

1 void insertAtTail(string text, string user) {
2     TweetNode* newNode = new TweetNode(text, user);
3
4     if (head == nullptr) {           // List empty
```

```

5     head = tail = newNode;
6 } else {
7     tail->next = newNode;    // old tail now links to new node
8     tail = newNode;        // tail moves forward
9 }
10 size++;
11 }

```

8.3 Insert at Position k — $O(k)$

- $k = 0 \rightarrow$ same as insertAtHead() $\rightarrow O(1)$.
- $k = \text{size} \rightarrow$ same as insertAtTail() $\rightarrow O(1)$.
- $0 < k < \text{size} \rightarrow$ traverse $k-1$ steps to find the predecessor, then rewire 2 pointers.

Insert 'X' at position 2 $\rightarrow$ traverse to B, then rewire 2 pointers.

```

insertAtPosition.cpp // g++ -std=c++17
1 void insertAtPosition(int k, string text, string user) {
2     if (k <= 0 || head == nullptr) { insertAtHead(text, user); return; }
3     if (k >= size) { insertAtTail(text, user); return; }
4
5     TweetNode* curr = head;
6     for (int i = 0; i < k - 1; i++) // O(k-1) traversal
7         curr = curr->next;
8
9     TweetNode* newNode = new TweetNode(text, user);
10    newNode->next = curr->next; // 1. point forward
11    curr->next = newNode;      // 2. predecessor links to new
12    size++;
13 }

```

Trace

Insert **X** at pos 2 in $[A] \rightarrow [B] \rightarrow [C] \rightarrow [D]$: curr = B (after 1 step, $k-1=1$); X.next = C ; B.next = X. Result: $[A] \rightarrow [B] \rightarrow [X] \rightarrow [C] \rightarrow [D]$. Time: $O(2) = O(k)$.

9. Delete Operations

Deletion has three classical cases — head, tail and arbitrary node by value. Each case differs in how many pointers must be updated and whether the tail pointer needs fixing.

9.1 Delete Head — $O(1)$

```
deleteHead.cpp // g++ -std=c++17
1 void deleteHead() {
2     if (!head) return;           // empty list
3     TweetNode* temp = head;
4     head = head->next;           // advance head
5     if (!head) tail = nullptr;   // list became empty
6     delete temp;                 // free memory
7     size--;
8 }
```

9.2 Delete Tail — $O(n)$ in Singly

Why $O(n)$? Because the tail node has no back-pointer in a singly list, we must scan from head to find the second-last node and update its next to NULL.

```
deleteTail.cpp // g++ -std=c++17
1 void deleteTail() {
2     if (!head) return;
3     if (head == tail) {           // single-node list
4         delete head; head = tail = nullptr; size--; return;
5     }
6     TweetNode* curr = head;
7     while (curr->next != tail)    // scan to second-last ->
8         curr = curr->next;         $O(n)$ 
9     delete tail;
10    tail = curr;
11    tail->next = nullptr;
12    size--;
13 }
```

9.3 Delete by Value — $O(n)$

- `prev = NULL; curr = head;`
- Traverse: while `curr != NULL && curr->data != target` → `prev = curr; curr = curr->next;`
- If `curr == NULL` → value not found → return.
- Otherwise `prev->next = curr->next;` — predecessor skips the target.
- If `curr == tail` → `tail = prev;`
- `delete curr; size--;`

```
deleteByValue.cpp // g++ -std=c++17
```

```
1 void deleteByValue(string target) {
2     if (!head) return;
3     if (head->text == target) { deleteHead(); return; }
4
5     TweetNode* prev = nullptr;
6     TweetNode* curr = head;
7     while (curr && curr->text != target) {
8         prev = curr;
9         curr = curr->next;
10    }
11    if (!curr) { cout << "Not found"; return; }
12
13    prev->next = curr->next;    // bypass the node
14    if (curr == tail) tail = prev;    // fix tail
15    delete curr;
16    size--;
17 }
```

Critical exam point

Always check `curr != nullptr` BEFORE accessing `curr->data`. Dereferencing a NULL pointer causes a segmentation fault — the #1 beginner error in linked-list programs.

10. Traversal — Forward & Backward

Traversal means visiting every node exactly once. In a singly linked list we can move only forward; in a doubly linked list we can move either way.

10.1 Forward Traversal (Singly & Doubly) — $O(n)$

```
traverse.cpp // g++ -std=c++17
1 void traverse() {
2     TweetNode* curr = head;
3     while (curr != nullptr) {           // CHECK first, then dereference
4         cout << "@" << curr->user << ": " << curr->text << " -> ";
5         curr = curr->next;
6     }
7     cout << "NULL" << endl;
8 }
```

10.2 Backward Traversal (Doubly only) — $O(n)$

```
traverseBackward.cpp // g++ -std=c++17
1 void traverseBackward() {
2     TweetNodeDL* curr = tail;
3     while (curr != nullptr) {
4         cout << curr->text << " <- ";
5         curr = curr->prev;           // follow PREV pointer
6     }
7     cout << "NULL" << endl;
8 }
9 // Output: T1 <- T2 <- T3 <- T4 <- NULL (oldest first if head is newest)
```

Quick revision

Traversal is the foundation of every other operation — search, count, print, reverse. If you can write a clean traversal loop, you are 50% done with linked lists.

11. Search Operations

Linked lists have no index formula. To find a node we start at head and examine each node one by one. Best case **O(1)** if the target is the head, worst case **O(n)** if it is the tail or absent.

```
searchByKeyword.cpp // g++ -std=c++17
1  bool searchByKeyword(string keyword) {
2      TweetNode* curr = head;
3      int pos = 0;
4      while (curr != nullptr) {
5          if (curr->text.find(keyword) != string::npos) {
6              cout << "Found at position " << pos
7                  << " by @" << curr->user << endl;
8              return true;
9          }
10         curr = curr->next;
11         pos++;
12     }
13     cout << "Keyword not found" << endl;
14     return false;
15 }
```

Why binary search fails on linked lists

- Binary search needs $O(1)$ random access — not available in a linked list.
- To reach the middle node we still need $O(n/2)$ traversal — which collapses back to $O(n)$.
- Therefore linked-list search is always linear $O(n)$.

12. Count Nodes

If we maintain a **size** counter and update it on every insert/delete, `count()` becomes $O(1)$. Without a counter, we must traverse — $O(n)$.

```
count.cpp // g++ -std=c++17
1 // O(n) version - traversal
2 int countNodes() {
3     int c = 0;
4     TweetNode* curr = head;
5     while (curr) { c++; curr = curr->next; }
6     return c;
7 }
8
9 // O(1) version - using maintained counter
10 int getSize() { return size; }
```

Design tip

In production code we always maintain a size field on the list class. The few extra `size++/size--` instructions are negligible compared to the cost of an $O(n)$ scan every time someone asks for the length.

13. Reverse a Linked List — $O(n)$ Time, $O(1)$ Space

Reversing flips all *next* pointers so the old tail becomes the new head. The trick is to use **three pointers** — *prev*, *curr*, *nextNode* — so we never lose our place.

Three-pointer algorithm

```
reverse.cpp // g++ -std=c++17
1 void reverse() {
2     TweetNode* prev = nullptr;
3     TweetNode* curr = head;
4     TweetNode* nextNode = nullptr;
5
6     tail = head;                // old head will be new tail
7
8     while (curr != nullptr) {
9         nextNode = curr->next;    // 1. save the next
10        curr->next = prev;        // 2. flip the link
11        prev = curr;             // 3. step prev forward
12        curr = nextNode;         // 4. step curr forward
13    }
14    head = prev;                 // prev is the new head
15 }
```

Trace on [T4]→[T3]→[T2]→[T1]→NULL

```
trace.txt
1 Iter 1: T4.next = NULL    prev=T4    curr=T3
2 Iter 2: T3.next = T4      prev=T3    curr=T2
3 Iter 3: T2.next = T3      prev=T2    curr=T1
4 Iter 4: T1.next = T2      prev=T1    curr=NULL (stop)
5 head = T1 -> [T1]->[T2]->[T3]->[T4]->NULL
6 Time:  $O(n)$     Space:  $O(1)$  - in-place
```

14. Doubly Linked List Operations

A doubly linked list stores an extra **prev** pointer in every node. This makes deletion of any node — including the tail — an $O(1)$ operation, and enables backward traversal.

DLL Node structure

```
DLLNode.cpp // g++ -std=c++17
1 struct TweetNodeDL {
2     string text;
3     string user;
4     TweetNodeDL* prev;
5     TweetNodeDL* next;
6
7     TweetNodeDL(string t, string u) {
8         text = t; user = u;
9         prev = nullptr;
10        next = nullptr;
11    }
12 };
13
14 // Memory Layout:
15 // NULL <- [T4] <-> [T3] <-> [T2] <-> [T1] -> NULL
16 //          ^ head                               ^ tail
```

14.1 Insert at Head — $O(1)$

```
DLL_insertAtHead.cpp // g++ -std=c++17
1 void insertAtHead(string t, string u) {
2     TweetNodeDL* newNode = new TweetNodeDL(t, u);
3     if (!head) { head = tail = newNode; size++; return; }
4
5     newNode->next = head; // 1. new -> old head
6     head->prev = newNode; // 2. old head -> new (DOUBLY key step)
7     head = newNode; // 3. advance head
8     size++;
9 }
```

14.2 Insert at Tail — $O(1)$

```
DLL_insertAtTail.cpp // g++ -std=c++17
1 void insertAtTail(string t, string u) {
2     TweetNodeDL* newNode = new TweetNodeDL(t, u);
3     if (!tail) { head = tail = newNode; size++; return; }
4
5     newNode->prev = tail;
6     tail->next = newNode;
```

```

7     tail          = newNode;
8     size++;
9 }

```

14.3 Delete Tail — $O(1)$ ★ key advantage

```

DLL_deleteTail.cpp // g++ -std=c++17
1 void deleteTail() {
2     if (!tail) return;
3     TweetNodeDL* temp = tail;
4     tail = tail->prev;           //  $O(1)$  - prev pointer does the
    work
5     if (tail) tail->next = nullptr; // new tail's next = NULL
6     else head = nullptr;         // list became empty
7     delete temp;
8     size--;
9 }

```

14.4 Delete Any Node N (pointer known) — $O(1)$

```

DLL_deleteNode.cpp // g++ -std=c++17
1 void deleteNode(TweetNodeDL* N) {
2     if (!N) return;
3     if (N == head) { deleteHead(); return; }
4     if (N == tail) { deleteTail(); return; }
5
6     N->prev->next = N->next;      // A skips over N -> points to B
7     N->next->prev = N->prev;      // B points back past N -> points to A
8     delete N;
9     size--;
10    // Only 2 pointer rewires -  $O(1)$  - NO SCANNING NEEDED
11 }

```

Why this matters

The same operation in a singly linked list takes $O(n)$ because we have to scan to find N's predecessor. In a doubly linked list, both neighbours are already known via $N->prev$ and $N->next$.

15. Complete Complexity Comparison

Operation	Singly — Time	Doubly — Time	Space	Notes
Insert at head	$O(1)$	$O(1)$	$O(1)$	Always constant
Insert at tail	$O(1)^*$	$O(1)$	$O(1)$	* with tail pointer
Insert at position k	$O(k)$	$O(k)$	$O(1)$	Traverse k-1 then rewire
Delete head	$O(1)$	$O(1)$	$O(1)$	
Delete tail	$O(n)$	$O(1)$	$O(1)$	DLL wins big
Delete by value	$O(n)$	$O(n)$	$O(1)$	Linear search
Forward traversal	$O(n)$	$O(n)$	$O(1)$	
Backward traversal	—	$O(n)$	$O(1)$	DLL only
Search by keyword	$O(n)$	$O(n)$	$O(1)$	No binary search possible
Count nodes	$O(1)/O(n)$	$O(1)/O(n)$	$O(1)$	$O(1)$ with counter
Reverse	$O(n)$	$O(n)$	$O(1)$	In-place 3-pointer technique

Memory comparison

Variant	Bytes per node (typical 64-bit)	Per 1,000,000 tweets
Singly Linked List	data + 8 (next)	≈ 8 MB extra
Doubly Linked List	data + 16 (prev + next)	≈ 16 MB extra

16. Main Project — MainProjectLinkedList.cpp

This is the integrated, menu-driven console program that ties together every operation discussed so far. The user can post tweets, delete them, search by hashtag, view the timeline forwards or backwards, and switch between singly/doubly modes — all from a single interface.

Source code (1/2)

```
MainProjectLinkedList.cpp // g++ -std=c++17
1  #include <iostream>
2  #include <string>
3  using namespace std;
4
5  struct TweetNode {
6      string text, user;
7      TweetNode *prev, *next;
8      TweetNode(string t, string u) : text(t), user(u), prev(nullptr),
next(nullptr) {}
9  };
10
11 class TwitterTimeline {
12     TweetNode *head = nullptr, *tail = nullptr;
13     int size = 0;
14 public:
15     void postTweet(string t, string u) {                // insert at head
16         TweetNode* n = new TweetNode(t, u);
17         if (!head) head = tail = n;
18         else { n->next = head; head->prev = n; head = n; }
19         size++;
20         cout << "Posted by @" << u << endl;
21     }
22     void deleteLatest() {                                // delete head
23         if (!head) return;
24         TweetNode* t = head;
25         head = head->next;
26         if (head) head->prev = nullptr; else tail = nullptr;
27         delete t; size--;
28     }
29     void deleteOldest() {                                // delete tail O(1)
30         if (!tail) return;
31         TweetNode* t = tail;
32         tail = tail->prev;
33         if (tail) tail->next = nullptr; else head = nullptr;
34         delete t; size--;
35     }
36     void showFeed() {                                    // forward
traversal
37         cout << "\n--- TIMELINE (newest first) ---\n";
```

```

38     for (TweetNode* c = head; c; c = c->next)
39         cout << " * @" << c->user << ": " << c->text << endl;
40 }
41 void showOldestFirst() {                                // backward
traversal
42     cout << "\n--- THREAD (oldest first) ---\n";
43     for (TweetNode* c = tail; c; c = c->prev)
44         cout << " * @" << c->user << ": " << c->text << endl;
45 }
46 void searchHashtag(string h) {
47     for (TweetNode* c = head; c; c = c->next)
48         if (c->text.find(h) != string::npos)
49             cout << "    @" << c->user << ": " << c->text << endl;
50 }
51 int count() { return size; }
52 };

```

Source code (2/2)

MainProjectLinkList.cpp // g++ -std=c++17

```

1  int main() {
2      TwitterTimeline tw;
3      int choice;
4      string text, user;
5
6      do {
7          cout << "\n===== TWITTER TIMELINE =====\n"
8              << "1. Post tweet          5. Show feed\n"
9              << "2. Delete latest tweet    6. Show oldest first\n"
10             << "3. Delete oldest tweet    7. Search by #hashtag\n"
11             << "4. Tweet count          0. Exit\n"
12             << "Choice: ";
13         cin >> choice; cin.ignore();
14
15         switch (choice) {
16             case 1: cout << "Tweet: "; getline(cin, text);
17                     cout << "User: "; getline(cin, user);
18                     tw.postTweet(text, user); break;
19             case 2: tw.deleteLatest(); break;
20             case 3: tw.deleteOldest(); break;
21             case 4: cout << "Total tweets: " << tw.count() << endl;
break;
22             case 5: tw.showFeed(); break;
23             case 6: tw.showOldestFirst(); break;
24             case 7: cout << "Hashtag: "; getline(cin, text);
25                     tw.searchHashtag(text); break;
26         }
27     } while (choice != 0);
28
29     cout << "Bye!\n";

```

```
30     return 0;
31 }
```

Sample run

output.txt

```
1  ===== TWITTER TIMELINE =====
2  Choice: 1
3  Tweet: Hello world! #firsttweet
4  User: deepak
5  Posted by @deepak
6
7  Choice: 1
8  Tweet: Linked lists are fun #DSA
9  User: rohan
10 Posted by @rohan
11
12 Choice: 5
13 --- TIMELINE (newest first) ---
14 * @rohan : Linked lists are fun #DSA
15 * @deepak: Hello world! #firsttweet
16
17 Choice: 7
18 Hashtag: #DSA
19 @rohan: Linked lists are fun #DSA
```

17. Singly Linked List Project — MyLinkedList.cpp

A focused implementation that demonstrates every singly-linked-list operation in isolation — useful for revision and viva.

Source code (1/2)

```
MyLinkedList.cpp // g++ -std=c++17
1  #include <iostream>
2  #include <string>
3  using namespace std;
4
5  struct Node {
6      string data;
7      Node* next;
8      Node(string d) : data(d), next(nullptr) {}
9  };
10
11 class SinglyList {
12     Node *head = nullptr, *tail = nullptr;
13     int size = 0;
14 public:
15     void insertAtHead(string d) {
16         Node* n = new Node(d);
17         if (!head) head = tail = n;
18         else { n->next = head; head = n; }
19         size++;
20     }
21     void insertAtTail(string d) {
22         Node* n = new Node(d);
23         if (!head) head = tail = n;
24         else { tail->next = n; tail = n; }
25         size++;
26     }
27     void insertAt(int k, string d) {
28         if (k <= 0) { insertAtHead(d); return; }
29         if (k >= size) { insertAtTail(d); return; }
30         Node* c = head;
31         for (int i = 0; i < k - 1; i++) c = c->next;
32         Node* n = new Node(d);
33         n->next = c->next; c->next = n;
34         size++;
35     }
36     void deleteHead() {
37         if (!head) return;
38         Node* t = head; head = head->next;
```

```

39         if (!head) tail = nullptr;
40         delete t; size--;
41     }
42     void deleteTail() {
43         if (!head) return;
44         if (head == tail) { delete head; head = tail = nullptr; size--;
return; }
45         Node* c = head;
46         while (c->next != tail) c = c->next;
47         delete tail; tail = c; tail->next = nullptr;
48         size--;
49     }
50     void deleteByValue(string v) {
51         if (!head) return;
52         if (head->data == v) { deleteHead(); return; }
53         Node *p = nullptr, *c = head;
54         while (c & (c && c->data != v) { p = c; c = c->next; }
55         if (!c) return;
56         p->next = c->next;
57         if (c == tail) tail = p;
58         delete c; size--;
59     }
60 };

```

Source code (2/2)

MyLinkedList.cpp // g++ -std=c++17

```

1 void traverse() {
2     for (Node* c = head; c; c = c->next)
3         cout << c->data << " -> ";
4     cout << "NULL\n";
5 }
6 bool search(string v) {
7     for (Node* c = head; c; c = c->next)
8         if (c->data == v) return true;
9     return false;
10 }
11 int count() { return size; }
12 void reverse() {
13     Node *prev = nullptr, *curr = head, *nx = nullptr;
14     tail = head;
15     while (curr) { nx = curr->next; curr->next = prev; prev = curr; curr
= nx; }
16     head = prev;
17 }
18 };
19
20 int main() {
21     SinglyList L;
22     L.insertAtHead("Tweet B");

```

```

23     L.insertAtHead("Tweet A");           // newest on top
24     L.insertAtTail("Tweet C");
25     L.insertAt(1, "Tweet X");           // [A]->[X]->[B]->[C]
26     L.traverse();
27
28     cout << "Search Tweet B: " << (L.search("Tweet B") ? "YES" : "NO") <<
endl;
29     cout << "Count: " << L.count() << endl;
30
31     L.deleteByValue("Tweet B");
32     L.reverse();
33     L.traverse();
34     return 0;
35 }

```

Sample output

output.txt

```

1  Tweet A -> Tweet X -> Tweet B -> Tweet C -> NULL
2  Search Tweet B: YES
3  Count: 4
4  Tweet C -> Tweet X -> Tweet A -> NULL

```

18. Doubly Linked List Project — MyDoubly_linklist.cpp

This module showcases the unique strengths of a doubly linked list — **backward traversal** and $O(1)$ deletion of arbitrary nodes — by simulating a browsable tweet thread.

Source code (1/2)

```
MyDoubly_linklist.cpp // g++ -std=c++17
1  #include <iostream>
2  #include <string>
3  using namespace std;
4
5  struct DNode {
6      string data;
7      DNode *prev, *next;
8      DNode(string d) : data(d), prev(nullptr), next(nullptr) {}
9  };
10
11 class DoublyList {
12     DNode *head = nullptr, *tail = nullptr;
13     int size = 0;
14 public:
15     void insertAtHead(string d) {
16         DNode* n = new DNode(d);
17         if (!head) head = tail = n;
18         else { n->next = head; head->prev = n; head = n; }
19         size++;
20     }
21     void insertAtTail(string d) {
22         DNode* n = new DNode(d);
23         if (!tail) head = tail = n;
24         else { n->prev = tail; tail->next = n; tail = n; }
25         size++;
26     }
27     void deleteHead() {
28         if (!head) return;
29         DNode* t = head; head = head->next;
30         if (head) head->prev = nullptr; else tail = nullptr;
31         delete t; size--;
32     }
33     void deleteTail() {                // O(1) - the DLL highlight
34         if (!tail) return;
35         DNode* t = tail; tail = tail->prev;
36         if (tail) tail->next = nullptr; else head = nullptr;
37         delete t; size--;
38     }
```

```

39     void deleteNode(DNode* N) {           // O(1) when ptr known
40         if (!N) return;
41         if (N == head) { deleteHead(); return; }
42         if (N == tail) { deleteTail(); return; }
43         N->prev->next = N->next;
44         N->next->prev = N->prev;
45         delete N; size--;
46     }
47     void forward() {
48         for (DNode* c = head; c; c = c->next) cout << c->data << " <-> ";
49         cout << "NULL\n";
50     }
51     void backward() {
52         for (DNode* c = tail; c; c = c->prev) cout << c->data << " <-> ";
53         cout << "NULL\n";
54     }
55 };

```

Sample output

output.txt

```

1 Forward (newest -> oldest): T1 <-> T2 <-> T3 <-> T4 <-> NULL
2 Backward (oldest -> newest): T4 <-> T3 <-> T2 <-> T1 <-> NULL
3 After trimming both ends:    T2 <-> T3 <-> NULL

```

19. Real-World Applications

Application	Linked-list role	Variant
Twitter / Instagram feed	Posts ordered newest→oldest, instant insert/delete	Doubly
Web browser history	Back/forward navigation between pages	Doubly
Music player playlist	Next/previous track, dynamic queue	Doubly
LRU cache	Move-to-front on access, $O(1)$ eviction at tail	Doubly
Undo/Redo stack	Each action is a node; redo by walking forward	Doubly
Hash table chaining	Collision lists at each bucket	Singly
Polynomial arithmetic	Each term = node (coeff, exp)	Singly
Operating-system free list	List of free memory blocks	Singly
Image viewer slideshow	Cycle through photos forward/back	Doubly (circular)

20. Advantages & Disadvantages

Advantages

- Dynamic size — grows and shrinks at run-time without reallocation.
- $O(1)$ insertion / deletion at known positions (head, tail, given pointer).
- No memory wastage — exactly as many nodes as needed are allocated.
- Flexible structure — easy to extend into stacks, queues, deques, trees, graphs.
- Efficient for frequent insert/delete workloads such as feeds and event queues.

Disadvantages

- No random access — accessing the k -th node is $O(k)$.
- Extra memory per node for pointers (4 or 8 bytes each).
- Poor cache locality — scattered nodes cause cache misses.
- More complex code than arrays — pointer bugs (dangling, NULL deref) are common.
- Cannot use binary search even on sorted data.

When NOT to use a linked list

If your workload is mostly random index access (e.g., look-up by ID) or numerical computation on large blocks of data, prefer arrays or vectors. Cache locality alone often makes them 10× faster.

21. Conclusion

This project successfully implemented a **Twitter-Timeline Linked List Management System** in C++, covering both the **Singly** and **Doubly Linked List** variants. The three connected programs — `MainProjectLinkedList.cpp`, `MyLinkedList.cpp` and `MyDoubly_linklist.cpp` — together demonstrate every fundamental linked-list operation with a real-world flavour.

The exercise reinforced several core ideas of the BCA Data-Structures syllabus:

- Pointers, dynamic memory allocation and self-referential structures.
- The trade-offs between arrays and linked lists.
- Why $O(1)$ insert at head and $O(1)$ delete at tail (DLL) are the perfect fit for a social-media timeline.
- Disciplined memory management — every new matched by a delete.
- Clean, modular C++ class design that mirrors real software engineering practice.

Future extensions could include a circular linked list for round-robin scheduling of trending topics, a skip-list for faster keyword search, and persistent storage of the timeline using file I/O. Even in its current form, the project is a compact yet complete demonstration of how a textbook data structure quietly powers a product that millions use every day.

Final note

Linked lists are not just an academic exercise — they are the silent backbone of feeds, caches, browsers, players and operating-system internals. Understanding them well is the first true step into the world of advanced data structures.

22. Bibliography / References

- Horowitz, Sahni & Anderson-Freed — Fundamentals of Data Structures in C++, Universities Press.
- Robert Lafore — Data Structures and Algorithms in C++, Sams Publishing.
- Yashavant Kanetkar — Data Structures Through C++, BPB Publications.
- Bjarne Stroustrup — The C++ Programming Language (4th Edition), Addison-Wesley.
- Cormen, Leiserson, Rivest & Stein — Introduction to Algorithms (CLRS), MIT Press.
- GeeksforGeeks — Linked-List tutorials & problems — <https://www.geeksforgeeks.org/data-structures/linked-list/>
- TutorialsPoint — Data Structures — https://www.tutorialspoint.com/data_structures_algorithms/linked_list_algorithms.htm
- cppreference.com — <https://en.cppreference.com/>
- BCA Semester-II Syllabus — Data Structures with C++, University curriculum (2024-25).

— End of Project Report —

*Twitter Timeline · Linked List Management System in C++
Submitted by Divyansh · BCA Semester II*

A. Appendix — Source Code Index

File	Description	Chapter
MainProjectLinkedList.cpp	Menu-driven Twitter-timeline simulator (DLL based)	16
MyLinkedList.cpp	Singly Linked List — all operations in isolation	17
MyDoubly_linklist.cpp	Doubly Linked List — backward traversal, O(1) deletions	18

PROJECT REPORT ON
TWITTER TIMELINE
LINKED LIST MANAGEMENT SYSTEM

THANK
YOU!



Thank you for taking the time to
review this project report.

— ♥ —
Your feedback is highly appreciated!